DNS Security – Assignment 2

23b0976 & 23b1063

1 Phase 1: Baseline DNS

1.1 Objective

Phase 1 provides the baseline plain-DNS deployment. The client uses the recursive resolver, which obtains records from the authoritative hierarchy. DNSSEC validation is disabled. The client can reach the resolver network but cannot directly query the authoritative network.

1.2 Deployment and verification

```
cd dns_security/phase1
docker compose build
docker compose up -d
./routers/configure-all-routes.sh
docker compose exec -T client dig @172.16.2.10 example.com
docker compose exec -T client curl -s http://example.com
docker compose exec -T client dig +time=1 +tries=1 @172.16.3.10 example.com
```

The first DNS query returns `172.16.1.20`, and the HTTP request returns the legitimate web page. The direct query to the root authoritative server fails because the authoritative subnet is isolated from the client.

```
sanjanaboyina@Sanjanas-MacBook-Air phase1 % docker compose exec -T client dig @172.16.2.10 example.com

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> @172.16.2.10 example.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 5654
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: b4f004ae85e6dbf7010000006ada4ce18b82cc1a369f35e6 (good)
;; QUESTION SECTION:
;example.com.                   IN      A

;; ANSWER SECTION:
example.com.            85680   IN      A       172.16.1.20

;; Query time: 1 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Thu Oct 22 17:50:25 UTC 2026
;; MSG SIZE  rcvd: 84

sanjanaboyina@Sanjanas-MacBook-Air phase1 % docker compose exec -T client curl -s http://example.com
<!DOCTYPE html>
<html>
<head>
    <title>Legitimate example.com</title>
</head>
<body>
    <h1>Legitimate example.com</h1>
    <p>Served by the real webserver at 172.16.1.20.</p>
    <p>CS6008 Network Security - DNS Security Assignment, Phase 1.</p>
</body>
</html>
sanjanaboyina@Sanjanas-MacBook-Air phase1 %
```

Figure 1: The legitimate address 172.16.1.20 and the legitimate web page

```
sanjanaboyina@Sanjanas-MacBook-Air phase1 % docker compose exec -T client dig +time=1 +tries=1 @172.16.3.10 example.com
;; communications error to 172.16.3.10#53: timed out

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> +time=1 +tries=1 @172.16.3.10 example.com
; (1 server found)
;; global options: +cmd
;; no servers could be reached

What's next:
    Debug this Compose error with Gordon → docker ai "help me fix this compose error"
sanjanaboyina@Sanjanas-MacBook-Air phase1 %
```

Figure 2: A timeout or failure, demonstrating that the client cannot directly reach the authoritative subnet.

1.3 DNS Packet Capture

A packet capture was performed on the core-router while the resolver looked up `example.com`. The capture shows the iterative DNS resolution process through the root, TLD, and authoritative domain servers.

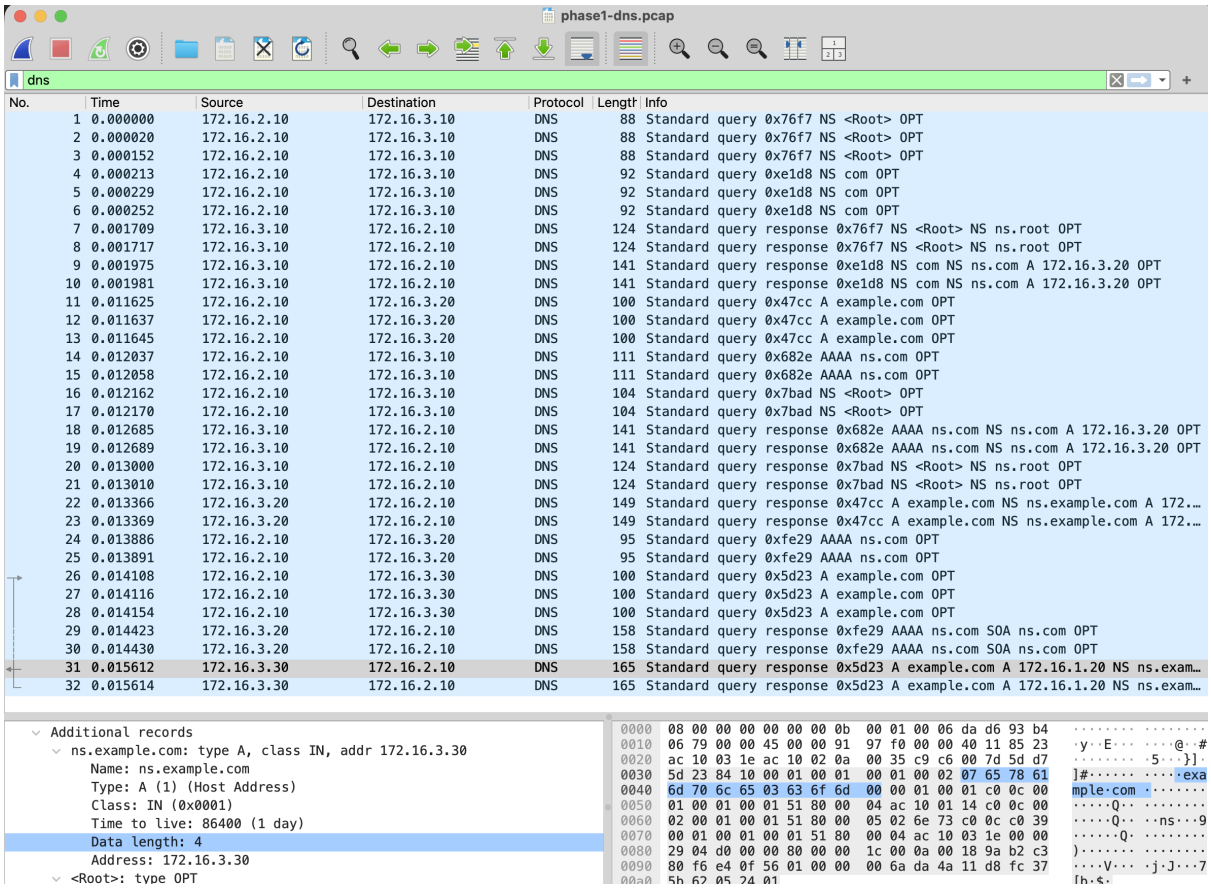


Figure 3: Wireshark capture showing DNS queries and responses between the resolver and the root, TLD, and authoritative domain servers.

Root server. The resolver at `172.16.2.10` queries the root DNS server at `172.16.3.10`. The root response provides referral information directing the resolver towards the `.com` TLD server at `172.16.3.20`.

TLD server. The resolver queries the TLD server at `172.16.3.20`. The response refers the resolver to the authoritative nameserver for `example.com`, at `172.16.3.30`.

Authoritative domain server. The resolver queries `172.16.3.30` for the A record of `example.com`. The response contains the IPv4 address `172.16.1.20`, identifying the legitimate webserver.

```
sanjanaboyina@Sanjanas-MacBook-Air phase1 % docker compose exec -T resolver ip route
default via 172.16.2.1 dev eth0
172.16.1.0/24 via 172.16.2.2 dev eth0
172.16.2.0/24 dev eth0 scope link  src 172.16.2.10
172.16.3.0/24 via 172.16.2.3 dev eth0
sanjanaboyina@Sanjanas-MacBook-Air phase1 % docker compose exec -T core-router ip route
default via 172.16.3.1 dev eth0
172.16.1.0/24 via 172.16.2.2 dev eth1
172.16.2.0/24 dev eth1 proto kernel scope link src 172.16.2.3
172.16.3.0/24 dev eth0 proto kernel scope link src 172.16.3.3
sanjanaboyina@Sanjanas-MacBook-Air phase1 %
```

Figure 4: Routing tables of the resolver and core router showing the configured static routes.

Routing. The resolver uses its static route for `172.16.3.0/24` via `172.16.2.3`, the core-router's resolver-side interface. The core-router forwards the query towards the authoritative subnet through its `172.16.3.3` interface. This allows the resolver to reach the DNS hierarchy without direct connectivity from the client to the authoritative subnet.

2 Phase 2: On-Path DNS Response Forgery

2.1 Objective

The attacker is placed on the resolver-to-authoritative path. It intercepts DNS responses with a libpcap program. In relay mode it forwards packets unchanged. In forge mode it changes the `A` record for `example.com` from the legitimate webserver to the rogue webserver. Other names are relayed without modification.

2.2 Attacker Placement and Traffic Diversion

The attacker is placed on the resolver subnet at `172.16.2.100`. A diversion route is configured on the core-router so that DNS responses destined for the resolver at `172.16.2.10` pass through the attacker. The interceptor then forwards the responses to the resolver, either unchanged or with the target A record modified. The resolver and authoritative nameservers retain their Phase 1 configuration.

2.3 Transparent relay

```
cd dns_security/phase2
docker compose up --build -d
./routers/configure-all-routes.sh
./scripts/setup-phase2-diversion.sh
./scripts/run-interceptor-relay.sh
docker compose exec -T resolver rndc flush
docker compose exec -T client dig @172.16.2.10 example.com
docker compose exec -T client curl -s http://example.com
```

The answer is `172.16.1.20`, and the legitimate page is returned.

```
[sanjanaboyina@Sanjanas-MacBook-Air phase2 % ./routers/configure-all-routes.sh
Static routes configured.
[sanjanaboyina@Sanjanas-MacBook-Air phase2 % ./scripts/setup-phase2-diversion.sh
Diversion route installed:
172.16.2.10 via 172.16.2.100 dev eth1 src 172.16.2.3 uid 0
    cache
[sanjanaboyina@Sanjanas-MacBook-Air phase2 % ./scripts/run-interceptor-relay.sh
Stopping any running interceptor
Starting interceptor in TRANSPARENT RELAY mode
Interface: eth0
Attacker MAC: 2e:51:11:6c:ec:2b
Resolver MAC: 36:c7:c6:15:ab:06
Mode: TRANSPARENT RELAY
Listening for DNS responses destined for 172.16.2.10...
[sanjanaboyina@Sanjanas-MacBook-Air phase2 % docker compose exec -T resolver rndc flush
[sanjanaboyina@Sanjanas-MacBook-Air phase2 % docker compose exec -T client dig @172.16.2.10 example.com

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> @172.16.2.10 example.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 52045
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 73e45ed81b1a653c010000006ada5bfcad6e02b131f17f35 (good)
;; QUESTION SECTION:
;example.com.                   IN      A

;; ANSWER SECTION:
example.com.            86400   IN      A       172.16.1.20

;; Query time: 116 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Thu Oct 22 18:54:52 UTC 2026
;; MSG SIZE  rcvd: 84

[sanjanaboyina@Sanjanas-MacBook-Air phase2 % docker compose exec -T client curl -s http://example.com
<!DOCTYPE html>
<html>
<head>
    <title>Legitimate example.com</title>
</head>
<body>
    <h1>Legitimate example.com</h1>
    <p>Served by the real webserver at 172.16.1.20.</p>
    <p>CS6008 Network Security - DNS Security Assignment, Phase 1.</p>
</body>
</html>
```

Figure 5: The unmodified DNS answer and legitimate page while the attacker is relaying.

2.4 Forgery attack

```
./scripts/run-interceptor-forge.sh
docker compose exec -T resolver rndc flush
docker compose exec -T client dig @172.16.2.10 example.com
docker compose exec -T client curl -s http://example.com
```

The answer changes to `172.16.1.30`, and the rogue page is returned. This succeeds because plain DNS has no cryptographic authentication or integrity protection.

```
sanjanaboyina@Sanjanas-MacBook-Air phase2 % ./scripts/run-interceptor-forge.sh
Stopping any running interceptor
Starting interceptor in FORGING mode (FORGE=1)
Interface: eth0
Attacker MAC: 2e:51:11:6c:ec:2b
Resolver MAC: 36:c7:c6:15:ab:06
Mode: FORGING ENABLED
Listening for DNS responses destined for 172.16.2.10...
sanjanaboyina@Sanjanas-MacBook-Air phase2 % docker compose exec -T resolver rndc flush
sanjanaboyina@Sanjanas-MacBook-Air phase2 % docker compose exec -T client dig @172.16.2.10 example.com

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> @172.16.2.10 example.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 28862
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 3b2fa71392f0ce6c010000006ada5c82957f10b9dfe84c08 (good)
;; QUESTION SECTION:
;example.com.                   IN      A

;; ANSWER SECTION:
example.com.            86400   IN      A       172.16.1.30

;; Query time: 12 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Thu Oct 22 18:57:06 UTC 2026
;; MSG SIZE  rcvd: 84

sanjanaboyina@Sanjanas-MacBook-Air phase2 % docker compose exec -T client curl -s http://example.com
<!DOCTYPE html>
<html>
<head><title>Rogue Website</title></head>
<body>
  <h1>Rogue Website</h1>
  <p>Phase 2 DNS response forgery succeeded.</p>
</body>
</html>
```

Figure 6: The forged address `172.16.1.30` and the rogue website response.

```
sanjanaboyina@Sanjanas-MacBook-Air phase2 % docker compose exec -T client curl -s http://example.com
<!DOCTYPE html>
<html>
<head><title>Rogue Website</title></head>
<body>
  <h1>Rogue Website</h1>
  <p>Phase 2 DNS response forgery succeeded.</p>
</body>
</html>
sanjanaboyina@Sanjanas-MacBook-Air phase2 % docker compose exec -T resolver rndc dumpdb -cache
sanjanaboyina@Sanjanas-MacBook-Air phase2 % docker compose exec -T resolver sh -c \
  'grep -n -A 8 "example.com" /var/cache/bind/named_dump.db'
14:example.com.         86390   NS      ns.example.com.
15-; answer
16-                     10795   \-AAAA  ;-$NXRRSET
17:; example.com. SOA ns.example.com. hostmaster.example.com. 2026102101 3600 1800 604800 86400
18-; authanswer
19-                     86390   A       172.16.1.30
20-; glue
21:ns.example.com.              86390   A       172.16.3.30
22-; answer
23-ns.com.                      10790   \-AAAA  ;-$NXRRSET
24-; com. SOA ns.com. hostmaster.com. 2026102101 3600 1800 604800 86400
25-; glue
26-                     86390   A       172.16.3.20
27-;
28-; Address database dump
29-;
--
33:; ns.example.com. [v4 TTL 50] [v4 success] [v6 unexpected]
34-;    172.16.3.30 [srtt 1140] [flags 00000004] [edns 2/0] [plain 0/0] [udpsize 512] [cookie=b681744c5c1038a3010000006ada5ec02dcca8d65ab130a1] [ttl 50]
35-; ns.com. [v4 TTL 50] [v6 TTL 10790] [v4 not_found] [v6 nxrrset]
36-;    172.16.3.20 [srtt 1620] [flags 00000004] [edns 2/0] [plain 0/0] [udpsize 512] [cookie=2ae267665ab623eb010000006ada5ebb79fb18dff64a648f] [ttl 50]
37-;
38-; Unassociated entries
39-;
40-;    172.16.3.10 [srtt 1504] [flags 00000004] [edns 4/0] [plain 0/0] [udpsize 512] [cookie=e4b2c87abf85bd68010000006ada5ebb0ba1780d53ecd32e] [ttl 50]
41-;
```

Figure 7: Successful DNS response forgery: the resolver returns `172.16.1.30`, the rogue website is served, and the BIND cache dump contains the forged A record.

```
sanjanaboyina@Sanjanas-MacBook-Air phase2 % docker compose exec -T resolver \
[  dig @127.0.0.1 example.com +noall +answer
example.com.            86259   IN      A       172.16.1.30
sanjanaboyina@Sanjanas-MacBook-Air phase2 % ./scripts/run-interceptor-relay.sh

Stopping any running interceptor
Starting interceptor in TRANSPARENT RELAY mode
Interface: eth0
Attacker MAC: 2e:51:11:6c:ec:2b
Resolver MAC: 36:c7:c6:15:ab:06
Mode: TRANSPARENT RELAY
Listening for DNS responses destined for 172.16.2.10...
sanjanaboyina@Sanjanas-MacBook-Air phase2 % docker compose exec -T client \
[  dig @172.16.2.10 example.com +noall +answer
example.com.            86014   IN      A       172.16.1.30
sanjanaboyina@Sanjanas-MacBook-Air phase2 % docker compose exec -T resolver rndc flush

sanjanaboyina@Sanjanas-MacBook-Air phase2 % docker compose exec -T client \
[  dig @172.16.2.10 example.com +noall +answer
example.com.            86400   IN      A       172.16.1.20
sanjanaboyina@Sanjanas-MacBook-Air phase2 % docker compose exec -T client \
[  curl --noproxy '*' -sS http://example.com
<!DOCTYPE html>
<html>
<head>
    <title>Legitimate example.com</title>
</head>
<body>
    <h1>Legitimate example.com</h1>
    <p>Served by the real webserver at 172.16.1.20.</p>
    <p>CS6008 Network Security - DNS Security Assignment, Phase 1.</p>
</body>
</html>
sanjanaboyina@Sanjanas-MacBook-Air phase2 %
```

Figure 8: Cache persistence after switching to relay mode, followed by successful recovery of the legitimate DNS answer after flushing the resolver cache.

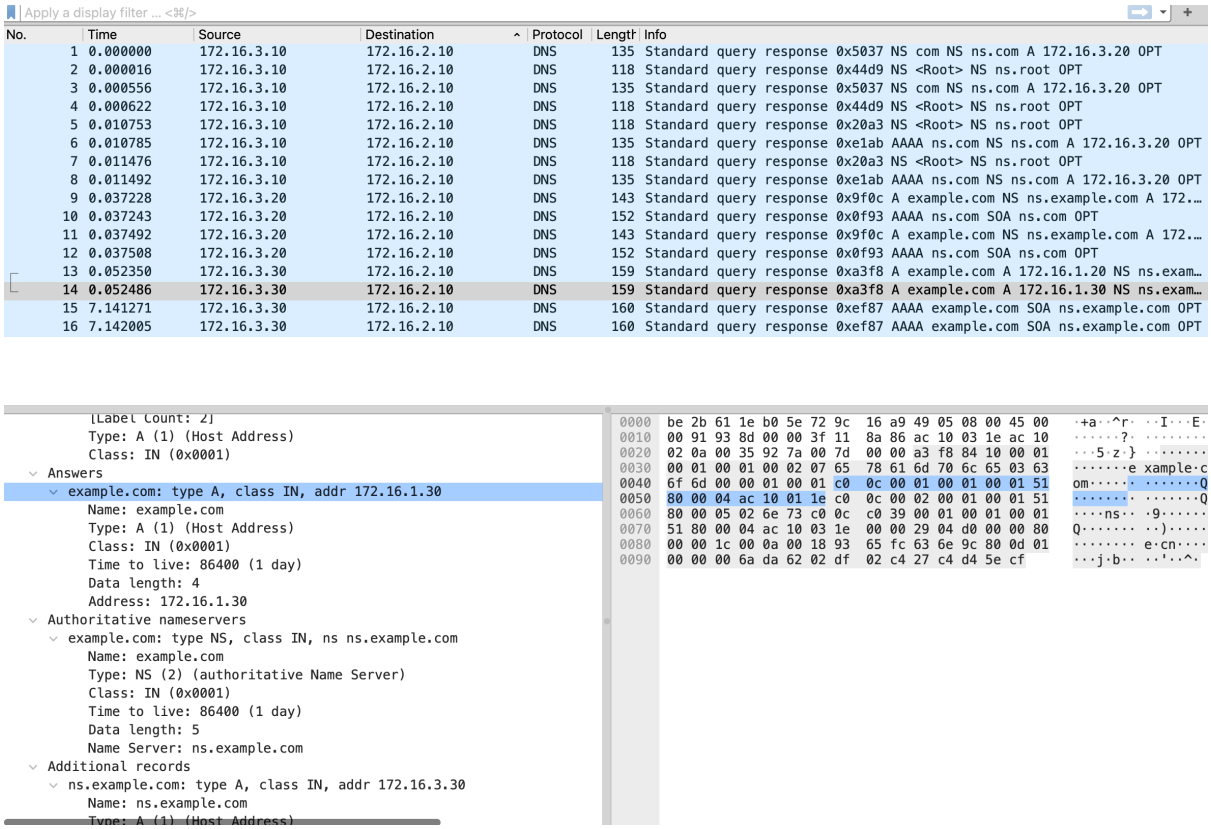


Figure 9: Packet capture showing the legitimate DNS answer `172.16.1.20` and the forged response containing `172.16.1.30` for `example.com`, sent toward the resolver at `172.16.2.10`.

2.5 Non-target lookup

```
docker compose exec -T client dig @172.16.2.10 www.example.com
docker compose exec -T client curl -s http://www.example.com
```

The non-target name still resolves to `172.16.1.20`. The interceptor only modifies the matching `example.com A` question.

```
</html>
sanjanaboyina@Sanjanas-MacBook-Air phase2 % docker compose exec -T client dig @172.16.2.10 www.example.com

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> @172.16.2.10 www.example.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 41328
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: b8d014ed9af3a5ff010000006ada5d328c601c391f0a5cb0 (good)
;; QUESTION SECTION:
;www.example.com.               IN      A

;; ANSWER SECTION:
www.example.com.        86400   IN      A       172.16.1.20

;; Query time: 7 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Thu Oct 22 19:00:02 UTC 2026
;; MSG SIZE  rcvd: 88

sanjanaboyina@Sanjanas-MacBook-Air phase2 % docker compose exec -T client curl -s http://www.example.com
<!DOCTYPE html>
<html>
<head>
    <title>Legitimate example.com</title>
</head>
<body>
    <h1>Legitimate example.com</h1>
    <p>Served by the real webserver at 172.16.1.20.</p>
    <p>CS6008 Network Security - DNS Security Assignment, Phase 1.</p>
</body>
</html>
sanjanaboyina@Sanjanas-MacBook-Air phase2 %
```

Figure 10: A legitimate response for a non-target name.

2.6 Security Analysis

An on-path attacker can observe the DNS query, including its transaction ID, UDP ports, and queried domain name. Unlike an off-path attacker, it does not need to guess these values or compete blindly with the authoritative response. It can modify the response in transit before forwarding it to the resolver. Since plain DNS does not provide cryptographic authentication of the response, the resolver may accept and cache the forged address.

The interceptor modifies only A records for `example.com`. Other DNS responses are forwarded without changes, so lookups such as `www.example.com` continue to resolve normally.

3 Phase 3: DNSSEC Chain of Trust

3.1 Objective

Phase 3 extends the DNS security experiment by enabling DNSSEC validation for the root , `com`, and `example.com` zones. The objective is to determine whether DNSSEC prevents an on-path attacker from modifying a DNS response and redirecting the client to a rogue web server.

The zones are signed using `ECDSAP256SHA256` (algorithm 13). The recursive resolver uses the root Key Signing Key (KSK) as its trust anchor and validates DNS responses through the chain of trust.

3.2 Zone Signing and Chain of Trust

The signing script generates DNSSEC keys and signs the three zones. Each zone has a Zone Signing Key (ZSK) and a Key Signing Key (KSK). The public DNSKEY records are included in the respective zones, and the resulting signed zone files contain RRSIG records.

The chain of trust is established through DS records in the parent zones. The root zone publishes the DS record for `com`, while the `com` zone publishes the DS record for `example.com`. These DS records link the parent zones to the DNSKEY records of their respective child zones.

The resolver is configured with the root KSK as its trust anchor and has DNSSEC validation enabled. Starting from this trusted key, the resolver validates the chain of DS and DNSKEY records before accepting signed DNS data.

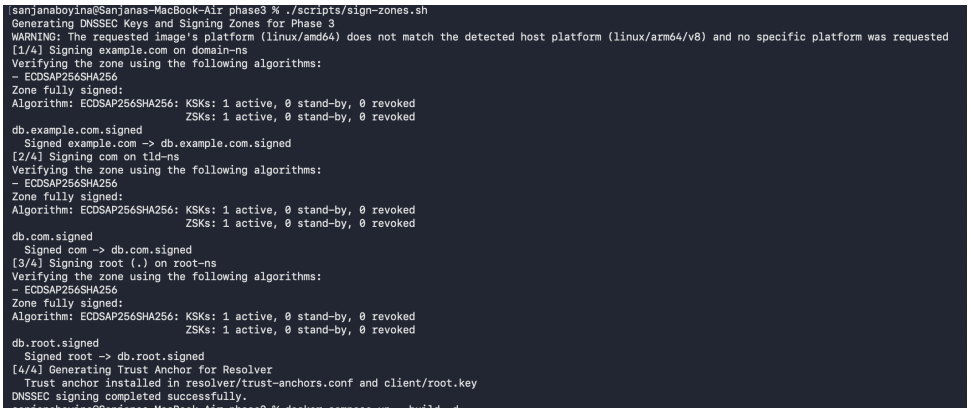


Figure 11: Successful DNSSEC key generation and signing of the `example.com`, `com`, and root zones.

3.3 Baseline DNSSEC Validation

Before enabling the attack, DNS resolution and HTTP access were tested without the interceptor in the path. The following commands were executed:

```
cd dns_security/phase3
./scripts/sign-zones.sh
docker compose up --build -d
./routers/configure-all-routes.sh

docker compose exec -T client \
  dig +dnssec @172.16.2.10 example.com

docker compose exec -T client \
  delv -a /etc/bind/root.key @172.16.2.10 example.com

docker compose exec -T client \
  curl -s http://example.com
```

The DNS query returned `NOERROR`, with the authenticated-data (`ad`) flag set. The answer contained the legitimate address `172.16.1.20` and an RRSIG record for the A record. The independent `delv` command reported that the answer was fully validated. The HTTP request also returned the legitimate website, confirming that normal DNS resolution and web access worked before the attack.

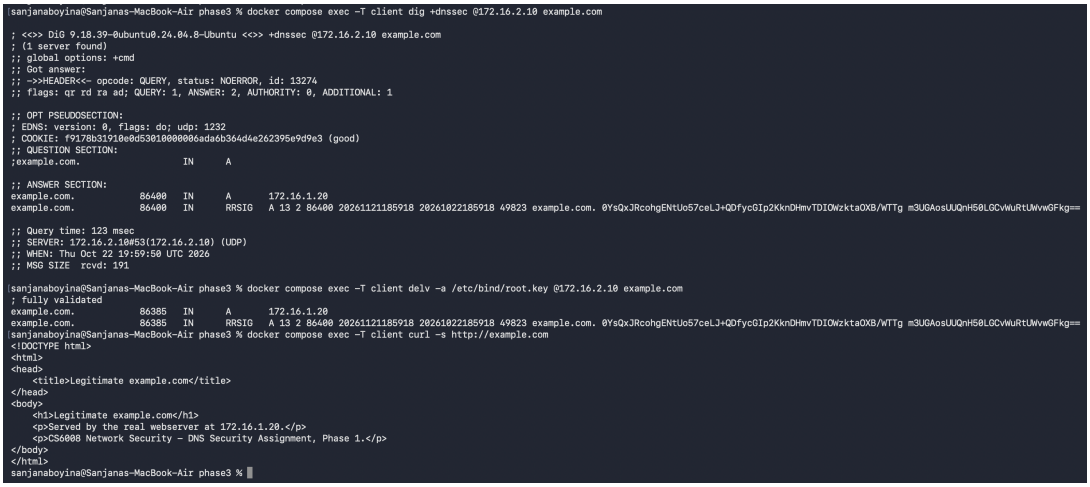


Figure 12: Baseline validation showing the authenticated-data flag, the RRSIG record, successful `delv` validation, and the legitimate website response.

3.4 Forged DNS Response Attack

The Phase 2 traffic-diversion setup and interceptor were reused without modifying the attacker binary. The interceptor was started in forging mode, and the resolver cache was flushed before issuing a fresh query.

```
./scripts/setup-diversion.sh
./scripts/run-interceptor-forge.sh

docker compose exec -T resolver rndc flush

docker compose exec -T client \
  dig @172.16.2.10 example.com

docker compose exec -T client \
  curl -v http://example.com

docker compose logs resolver | tail -n 10
```

The interceptor started successfully in forging mode and listened for DNS responses destined for the resolver at `172.16.2.10`. After the cache was flushed, the client query returned `SERVFAIL` with no answer, and the HTTP request failed because the hostname could not be resolved.

The resolver logs reported:

```
validating example.com/A: no valid signature found
RRSIG failed to verify resolving 'example.com/A/IN':
    172.16.3.30#53
```

These messages show that the resolver rejected the DNS response because the signature associated with the A record could not be validated. The resolver checks the RRSIG against the relevant DNSKEY and establishes trust in that key through the DS records and the configured root trust anchor. Modifying the signed A record without producing a valid signature causes validation to fail. Consequently, the resolver returns `SERVFAIL` rather than accepting the forged address.

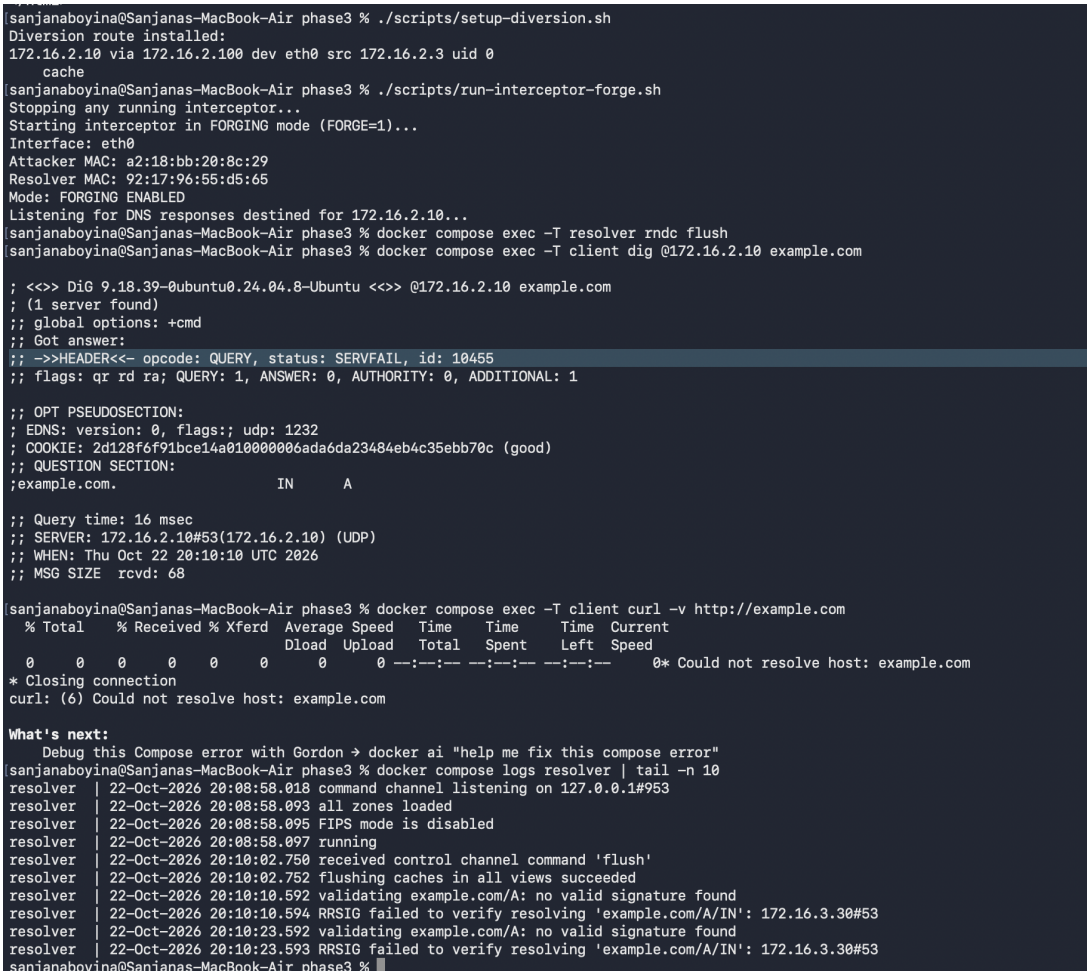


Figure 13: Forgery attack: the DNS query returns `SERVFAIL`, the HTTP request fails, and resolver logs report signature verification failure.

3.5 Packet Capture and Comparison with Phase 1

In Phase 1, DNS responses could be observed and modified by the on-path attacker. In Phase 2, the forged response could redirect the client because DNSSEC validation was not enabled. In Phase 3, the attacker can still observe and modify DNS traffic, but the resolver rejects forged data when its RRSIG fails verification.

The Wireshark capture shows DNS responses between the authoritative server (`172.16.3.30`) and the resolver (`172.16.2.10`). Comparing the original and modified A records, where visible, illustrates the attempted forgery. The resolver logs confirm that signature verification failed.

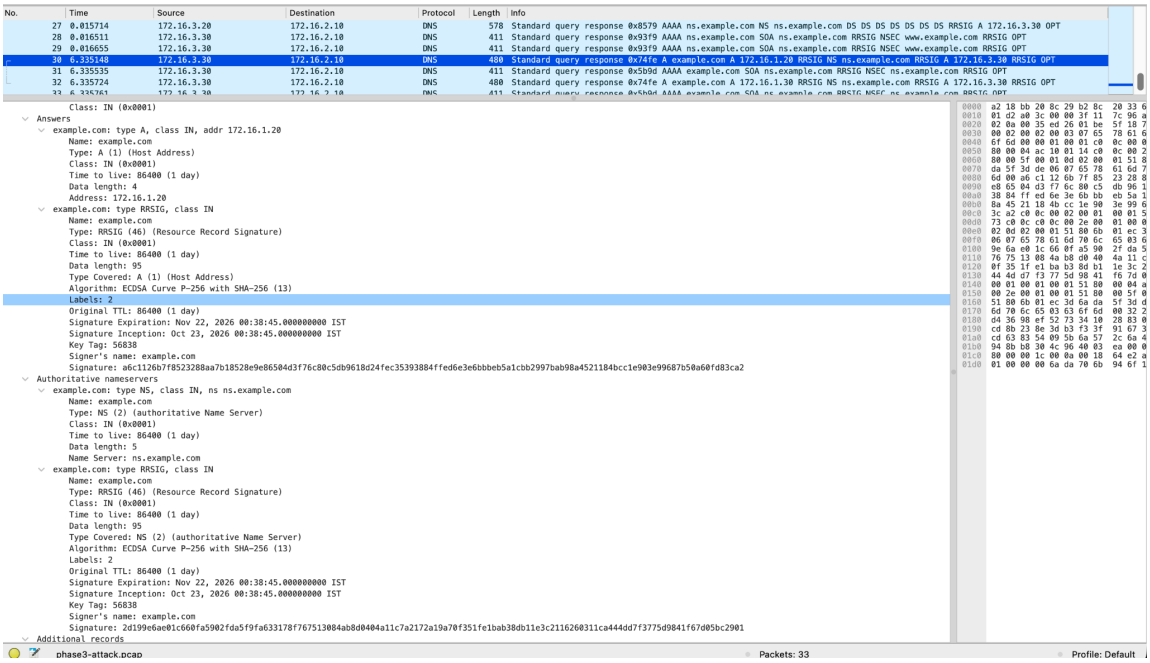


Figure 14: Original DNS response containing the legitimate A record and its RRSIG.

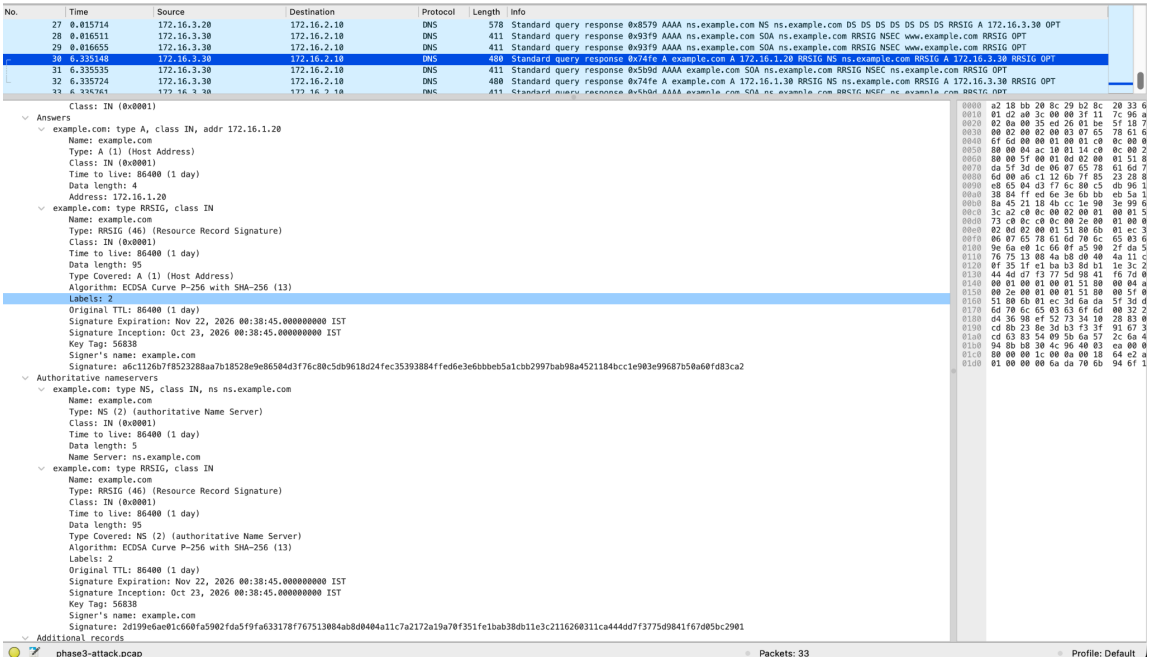


Figure 15: DNS response capture during the forgery attempt. Compare the A record and RRSIG with the original response.

DNSSEC provides authenticity, integrity, and authenticated denial of existence, but not confidentiality. An on-path attacker can still observe DNS queries, IP addresses, and DNSSEC records, or drop packets. Encryption is required to hide DNS traffic.

3.6 Signing Algorithm and Private-Key Handling

The zones use `ECDSAP256SHA256`, algorithm 13. ECDSA with the P-256 curve provides strong cryptographic security with relatively compact keys and signatures compared with typical RSA

configurations. Smaller signatures help reduce DNS response sizes and the risk of IP fragmentation. RSA remains widely supported, but RSA signatures are generally larger when configured for comparable security.

Private DNSSEC signing keys, generated signed zones, DS-set files, and compiled binaries must not be committed to the submitted repository. They are generated locally by the signing scripts, so the submission contains the source zones, configurations, and regeneration scripts instead.

4 Phase 4: Signature Stripping and Downgrade

4.1 Run A: Secure delegation

Run A contains a signed parent `com` zone with a valid DS record for `example.com`. The attacker strips signatures and forges the A record.

```
cd dns_security/phase4
./scripts/sign-zones-run-a.sh
docker compose up --build -d
./routers/configure-all-routes.sh
docker kill -s HUP root-ns tld-ns domain-ns
docker compose exec -T resolver rndc reconfig
docker compose exec -T resolver rndc flush
docker compose exec -T client dig +dnssec @172.16.2.10 example.com
./scripts/run-interceptor-strip.sh
./scripts/setup-diversion.sh
docker compose exec -T resolver rndc flush
docker compose exec -T client dig @172.16.2.10 example.com
docker compose exec -T client curl -v http://example.com
```

4.2 Run A: Secure Delegation

Run A uses a signed `example.com` zone whose DS record is published in the signed `com` zone. Before the attack, the resolver returns the legitimate address with the `ad` flag, confirming successful DNSSEC validation.

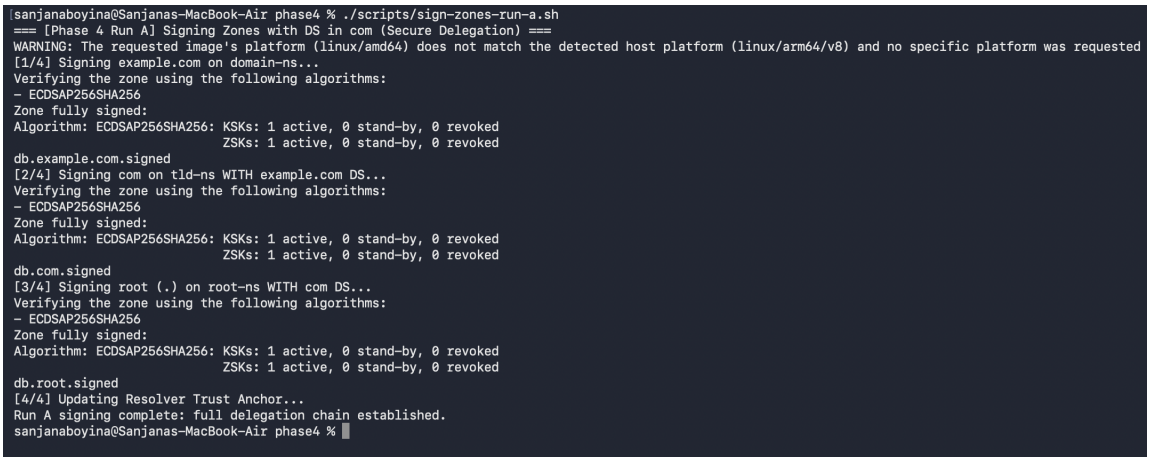


Figure 17: Signing the zones with a secure delegation: the parent publishes the DS record for `example.com`.

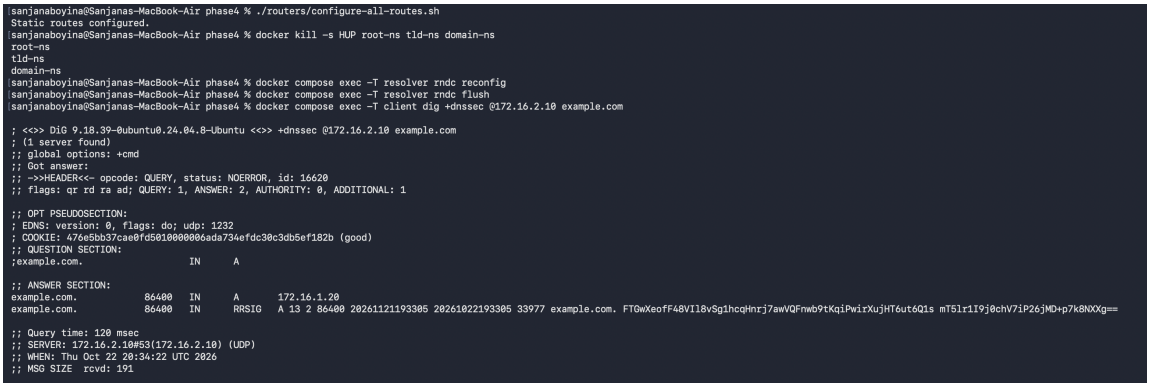


Figure 18: Baseline DNSSEC validation: the resolver returns `172.16.1.20`, with the `ad` flag and an RRSIG.

The attacker then strips DNSSEC records and injects the forged address `172.16.1.30`. The resolver returns `SERVFAIL`, and the HTTP request fails because the authenticated delegation prevents the attacker from making the zone appear unsigned.

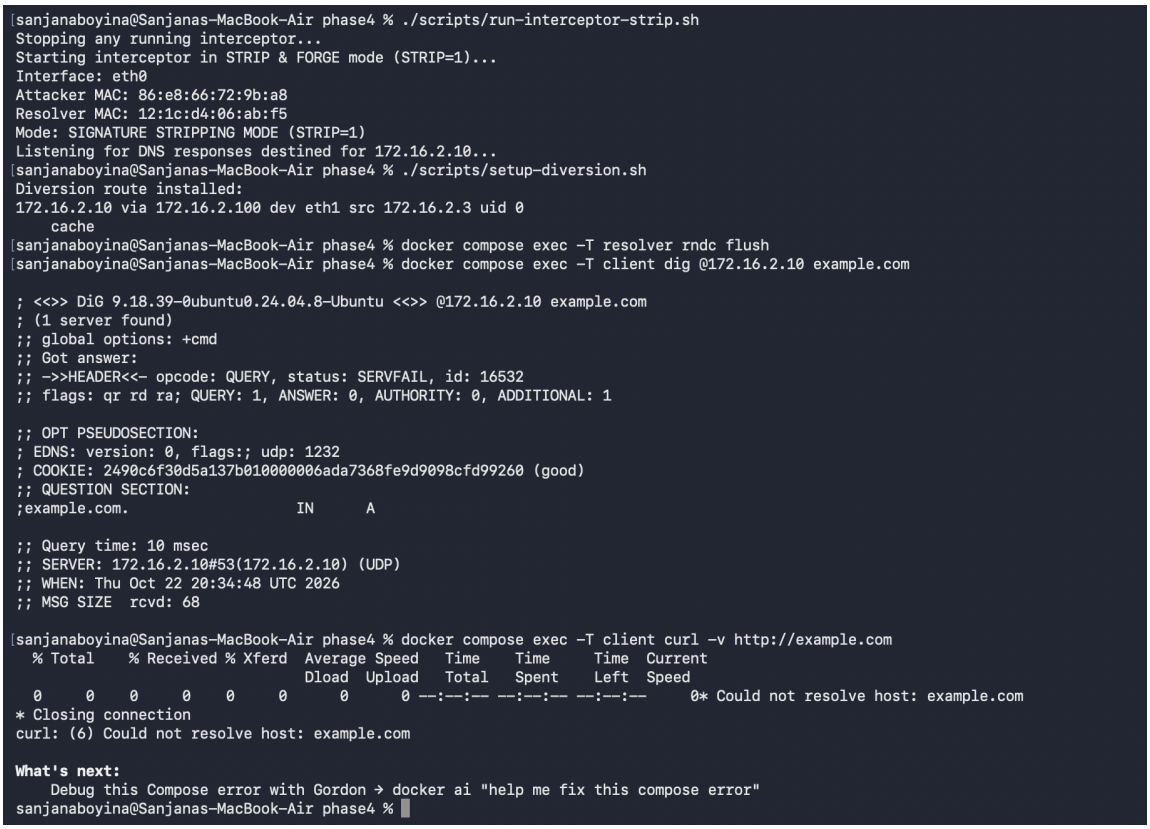


Figure 19: Run A attack: DNSSEC validation fails with `SERVFAIL`, and the client cannot resolve the website.

4.3 Run B: Missing parent DS

In Run B, the child is signed but the parent publishes no DS record. The resolver can authenticate the absence of the DS and correctly treats the delegation as insecure. The child signatures are then not anchored in the chain.

```
./scripts/sign-zones-run-b.sh
./scripts/teardown-diversion.sh
```

```
./scripts/stop-interceptor.sh
docker kill -s HUP root-ns tld-ns domain-ns
docker compose exec -T resolver rndc reconfig
docker compose exec -T resolver rndc flush
docker compose exec -T client dig @172.16.2.10 example.com
./scripts/run-interceptor-strip.sh
./scripts/setup-diversion.sh
docker compose exec -T resolver rndc flush
docker compose exec -T client dig @172.16.2.10 example.com
docker compose exec -T client curl -s http://example.com
```

4.4 Run B: Missing Parent DS

For Run B, the child zone remains signed, but the parent `com` zone no longer publishes a DS record for `example.com`. This removes the authenticated link between the parent and child.

```
sanjanaboyina@Sanjanas-MacBook-Air phase4 % ./scripts/sign-zones-run-b.sh

=== [Phase 4 Run B] Signing Zones WITHOUT example.com DS in com (Insecure Delegation) ===
WARNING: The requested image's platform (linux/amd64) does not match the detected host platform (linux/arm64/v8) and no specific platform was requested
[1/4] Signing example.com on domain-ns (child is fully signed)...
Verifying the zone using the following algorithms:
- ECDSAP256SHA256
Zone fully signed:
Algorithm: ECDSAP256SHA256: KSKs: 1 active, 0 stand-by, 0 revoked
                            ZSKs: 1 active, 0 stand-by, 0 revoked
db.example.com.signed
[2/4] Signing com on tld-ns WITHOUT example.com DS (operator omitted DS)...
Verifying the zone using the following algorithms:
- ECDSAP256SHA256
Zone fully signed:
Algorithm: ECDSAP256SHA256: KSKs: 1 active, 0 stand-by, 0 revoked
                            ZSKs: 1 active, 0 stand-by, 0 revoked
db.com.signed
[3/4] Signing root (.) on root-ns WITH com DS...
Verifying the zone using the following algorithms:
- ECDSAP256SHA256
Zone fully signed:
Algorithm: ECDSAP256SHA256: KSKs: 1 active, 0 stand-by, 0 revoked
                            ZSKs: 1 active, 0 stand-by, 0 revoked
db.root.signed
[4/4] Updating Resolver Trust Anchor...
Run B signing complete: example.com is signed, but parent com publishes NO DS.
sanjanaboyina@Sanjanas-MacBook-Air phase4 % ./scripts/teardown-diversion.sh
Diversion route removed. Current route to resolver:
172.16.2.10 dev eth1 src 172.16.2.3 uid 0
    cache
sanjanaboyina@Sanjanas-MacBook-Air phase4 % ./scripts/stop-interceptor.sh
Stopping interceptor on attacker...
Interceptor stopped.
sanjanaboyina@Sanjanas-MacBook-Air phase4 % docker kill -s HUP root-ns tld-ns domain-ns
root-ns
tld-ns
domain-ns
```

Figure 20: Run B signing: `example.com` is signed, but its DS record is omitted from the parent `com` zone.

Before the attack, the client can still resolve the legitimate address. However, the delegation is now insecure because the parent does not publish a DS record.

```
sanjanaboyina@Sanjanas-MacBook-Air phase4 % docker compose exec -T resolver rndc reconfig
sanjanaboyina@Sanjanas-MacBook-Air phase4 % docker compose exec -T resolver rndc flush
sanjanaboyina@Sanjanas-MacBook-Air phase4 % docker compose exec -T client dig @172.16.2.10 example.com

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> @172.16.2.10 example.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 3001
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 000e2d36afe55e1a010000006ada74a37c2da7ca45581110 (good)
;; QUESTION SECTION:
;example.com.                   IN      A

;; ANSWER SECTION:
example.com.            86400   IN      A       172.16.1.20

;; Query time: 14 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Thu Oct 22 20:40:03 UTC 2026
;; MSG SIZE  rcvd: 84
```

Figure 21: Run B baseline: the resolver returns the legitimate address `172.16.1.20`.

With signature stripping and response forgery enabled, the resolver accepts the forged address `172.16.1.30`, and the client receives the rogue website.

```
[sanjanaboyina@Sanjanas-MacBook-Air phase4 % ./scripts/run-interceptor-strip.sh
Stopping any running interceptor...
Starting interceptor in STRIP & FORGE mode (STRIP=1)...
Interface: eth0
Attacker MAC: 9a:6e:cb:aa:e7:94
Resolver MAC: 56:5c:4d:21:67:34
Mode: SIGNATURE STRIPPING MODE (STRIP=1)
Listening for DNS responses destined for 172.16.2.10...
[sanjanaboyina@Sanjanas-MacBook-Air phase4 % ./scripts/setup-diversion.sh
Diversion route installed:
172.16.2.10 via 172.16.2.100 dev eth1 src 172.16.2.3 uid 0
    cache
[sanjanaboyina@Sanjanas-MacBook-Air phase4 % docker compose exec -T resolver rndc flush
[sanjanaboyina@Sanjanas-MacBook-Air phase4 % docker compose exec -T client dig @172.16.2.10 example.com

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> @172.16.2.10 example.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 7480
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 0c1331d1ee186209010000006ada74c6d4ab1ae5fcfad395 (good)
;; QUESTION SECTION:
;example.com.                   IN      A

;; ANSWER SECTION:
example.com.            86400   IN      A       172.16.1.30

;; Query time: 11 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Thu Oct 22 20:40:38 UTC 2026
;; MSG SIZE  rcvd: 84

[sanjanaboyina@Sanjanas-MacBook-Air phase4 % docker compose exec -T client curl -s http://example.com
<!DOCTYPE html>
<html>
<head><title>Rogue Website</title></head>
<body>
  <h1>Rogue Website</h1>
  <p>Phase 2 DNS response forgery succeeded.</p>
</body>
</html>
sanjanaboyina@Sanjanas-MacBook-Air phase4 %
```

Figure 22: Run B attack succeeds: the resolver returns `172.16.1.30`, and the rogue website is served.

4.5 Conclusion

Downgrade resistance comes from the authenticated DS record in the parent zone, not merely from signing the child zone. In Run A, the valid DS record prevents signature stripping from bypassing validation. In Run B, omitting the DS record makes the delegation insecure and allows the forged response to be accepted. The operator mistake is signing the child zone without publishing its DS record in the parent.

5 Phase 5: DNSSEC Zone Walking

5.1 Objective

Standard NSEC records reveal a plaintext successor name. An attacker can query a nonexistent record, read the NSEC interval, and repeatedly follow successors until the zone is enumerated. This phase compares standard NSEC with NSEC3.

5.2 Why the zone walker is implemented in C

The assignment requires an attacker-side zone-walking demonstration. The program in `attacker/zonewalker.c` is intentionally self-contained: it sends DNS queries through libpcap, parses DNS denial-of-existence records, follows the plaintext successor in an NSEC record, and detects when the response contains NSEC3 instead. It prints the recovered names without making extra A-record queries.

The current file contains the complete raw-packet implementation. It does not need to be explained line by line; the important security logic is the NSEC successor-following loop and the NSEC3 detection.

1. construct a DNS query for a missing `TXT` record;
2. capture the matching DNS response and parse its sections;
3. read the plaintext "next domain name" from NSEC RDATA;
4. continue querying until the chain returns to the zone apex; and
5. stop and report protection when an NSEC3 record is detected.

The remaining code handles Ethernet/IP/UDP headers, checksums, ARP/MAC resolution, bounds checking, timeouts, and command-line defaults. These parts are required for a reliable raw-packet program, but are implementation details rather than separate security concepts.

5.3 NSEC walk

```
cd dns_security/phase5
./scripts/sign-walkable-zone.sh
docker compose build
docker compose up -d
docker compose restart root-ns tld-ns domain-ns resolver
./routers/configure-all-routes.sh
./scripts/run-zonewalker.sh
```

```
Static routes configured.
sanjanaboyina@Sanjanas-MacBook-Air phase5 % ./scripts/run-zonewalker.sh
Starting the Phase 5 zone walker...
================================================================
               PHASE 5: DNSSEC ZONE WALKER (C / libpcap)
================================================================
Target DNS Server: 172.16.2.10
Target Zone Apex:  example.com
Interface:         eth0
Attacker IP:       172.16.2.100

[+] Resolving MAC address for target 172.16.2.10...
[+] Target MAC: ce:96:c3:48:cc:46
[+] BPF filter active: udp and src host 172.16.2.10 and src port 53 and dst port 51973

--- Beginning Zone Walk Attack ---
[+] Query # 1: [example.com           ] -> NSEC next: [billing.example.com   ]
[+] Query # 2: [billing.example.com   ] -> NSEC next: [dev.example.com       ]
[+] Query # 3: [dev.example.com       ] -> NSEC next: [mail.example.com      ]
[+] Query # 4: [mail.example.com      ] -> NSEC next: [ns.example.com        ]
[+] Query # 5: [ns.example.com        ] -> NSEC next: [payroll.example.com   ]
[+] Query # 6: [payroll.example.com   ] -> NSEC next: [secret.example.com    ]
[+] Query # 7: [secret.example.com    ] -> NSEC next: [vpn.example.com       ]
[+] Query # 8: [vpn.example.com       ] -> NSEC next: [www.example.com       ]
[+] Query # 9: [www.example.com       ] -> NSEC next: [example.com           ]

[+] Full circle completed! Next name wrapped back to apex 'example.com'.

================================================================
                    ZONE WALK ATTACK SUMMARY
================================================================
Result:         SUCCESS (Zone Walk Complete via NSEC)
Total Queries:  9 (Efficiency constraint <= 9+2 queries satisfied)
Names Found:    9

No.   Domain Name
----------------------------------------
1     example.com
2     billing.example.com
3     dev.example.com
4     mail.example.com
5     ns.example.com
6     payroll.example.com
7     secret.example.com
8     vpn.example.com
9     www.example.com
================================================================
sanjanaboyina@Sanjanas-MacBook-Air phase5 %
```

Figure 23: A successful NSEC walk and the names discovered by the attacker.

Important implementation note. The walker uses a missing `TXT` query because the signed authoritative server returns an authenticated NSEC denial proof. The attack is not a signature forgery: the NSEC data is genuine, but its plaintext successor names disclose the zone ordering.

5.4 NSEC3 protection

```
./scripts/sign-fixed-zone.sh
docker compose restart root-ns tld-ns domain-ns resolver
./routers/configure-all-routes.sh
./scripts/run-zonewalker.sh
```

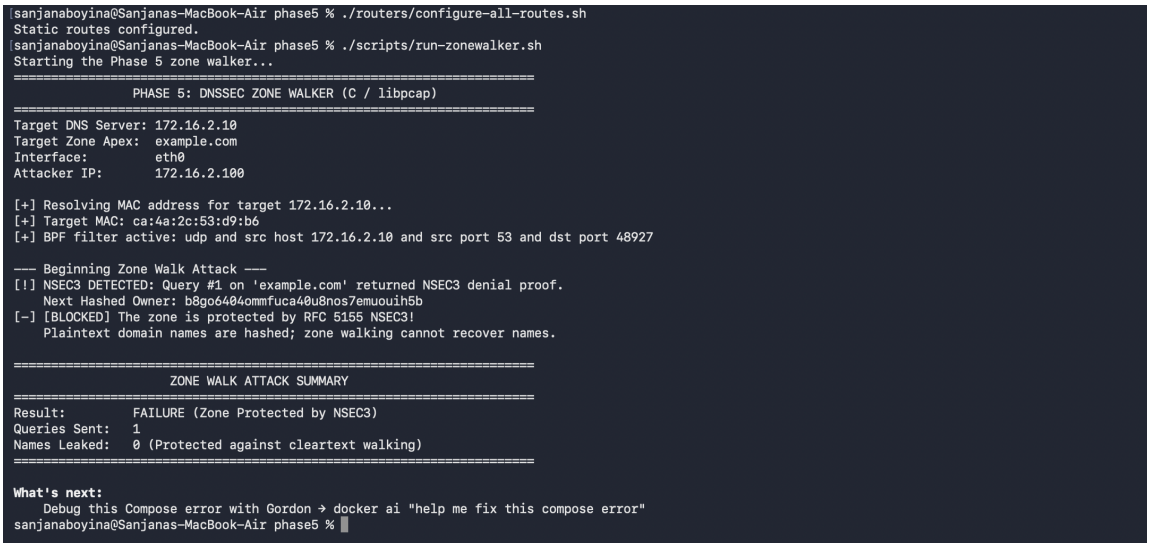


Figure 24: The NSEC3 result showing that no names were leaked by the simple walker.

5.5 Required Phase 5 files

The following files are sufficient for the Phase 5 experiment:

File	Purpose
`attacker/zonewalker.c`	Implements the NSEC walk and NSEC3 detection.
`attacker/Dockerfile`	Installs the compiler/libpcap and builds the walker.
`attacker/Makefile`	Provides the repeatable compilation command.
`scripts/run-zonewalker.sh`	Runs the compiled walker against the resolver.
`scripts/sign-walkable-zone.sh`	Generates the standard-NSEC test case.
`scripts/sign-fixed-zone.sh`	Generates the NSEC3 test case.
`scripts/sign-zones.sh`	Convenience alias for the fixed NSEC3 setup.
`routers/configure-all-routes.sh`	Makes the three Docker networks reachable.

The scripts are not separate attack mechanisms. They make the two test cases reproducible and avoid committing generated keys, signed zones, or compiled binaries.

5.6 Limitations

NSEC3 does not make names absolutely secret. An attacker who guesses a likely name may attempt a dictionary or brute-force attack against its hash. It prevents the direct efficient walk demonstrated in this phase.

6 Phase 6: Last-Mile DNS Response Forgery

6.1 Objective

The attacker is moved to the client subnet at `172.16.1.100`. The recursive resolver still validates DNSSEC correctly, but the final resolver-to-client DNS packet is ordinary cleartext DNS.

6.2 Deployment and transparent relay

```
cd dns_security/phase6
./scripts/sign-zones.sh
docker compose build
docker compose up -d
./routers/configure-all-routes.sh
./scripts/setup-diversion.sh
./scripts/run-interceptor-relay.sh
docker compose exec -T client dig @172.16.2.10 example.com
docker compose exec -T client curl -s http://example.com
docker compose exec -T client delv -a /etc/bind/root.key @172.16.2.10 example.com
```

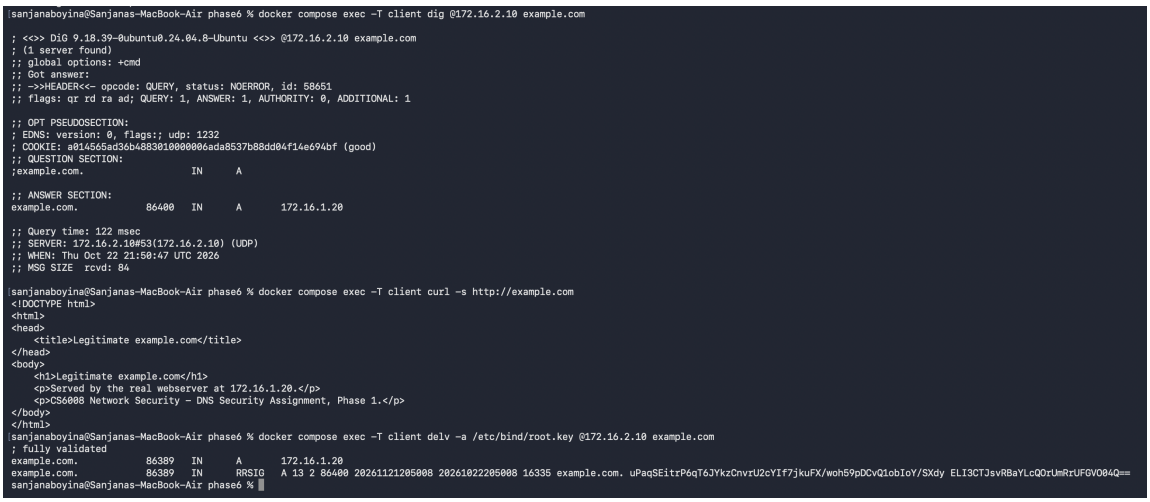


Figure 25: The legitimate result while the last-mile attacker is relaying.

6.3 Forgery against an ordinary stub

```
./scripts/run-interceptor-forge.sh
docker compose exec -T resolver rndc flush
docker compose exec -T client dig @172.16.2.10 example.com
docker compose exec -T client curl -s http://example.com
```

The ordinary stub accepts `172.16.1.30` and loads the rogue page. The `AD` bit is not independently trustworthy because it is part of the modifiable cleartext packet.

```
[sanjanaboyina@Sanjanas-MacBook-Air phase6 % ./scripts/run-interceptor-forge.sh
 Stopping any running interceptor...
 Starting interceptor in FORGING mode (FORGE=1)...
 Interface: eth0
 Attacker MAC: 0e:52:3e:fd:32:7e
 Client MAC: 52:77:7d:83:c1:bd
 Mode: FORGING ENABLED
 Listening for DNS responses destined for 172.16.1.10...
[sanjanaboyina@Sanjanas-MacBook-Air phase6 % docker compose exec -T resolver rndc flush
[sanjanaboyina@Sanjanas-MacBook-Air phase6 % docker compose exec -T client dig @172.16.2.10 example.com

 ; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> @172.16.2.10 example.com
 ; (1 server found)
 ;; global options: +cmd
 ;; Got answer:
 ;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 58570
 ;; flags: qr rd ra ad; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

 ;; OPT PSEUDOSECTION:
 ; EDNS: version: 0, flags:; udp: 1232
 ; COOKIE: 6384e3ea8ae01eb5010000006ada8572363dee6428fad6ba (good)
 ;; QUESTION SECTION:
 ;example.com.                   IN      A

 ;; ANSWER SECTION:
 example.com.            86400   IN      A       172.16.1.30

 ;; Query time: 6 msec
 ;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
 ;; WHEN: Thu Oct 22 21:51:46 UTC 2026
 ;; MSG SIZE  rcvd: 84

[sanjanaboyina@Sanjanas-MacBook-Air phase6 % docker compose exec -T client curl -s http://example.com
 <!DOCTYPE html>
 <html>
 <head><title>Rogue Website</title></head>
 <body>
   <h1>Rogue Website</h1>
   <p>Phase 2 DNS response forgery succeeded.</p>
 </body>
 </html>
 sanjanaboyina@Sanjanas-MacBook-Air phase6 %
```

Figure 26: The ordinary stub accepting the forged last-mile response.

6.4 Forgery against a locally validating client

```
docker compose exec -T client delv -a /etc/bind/root.key @172.16.2.10 example.com
```

The locally validating client rejects the changed A record because the original RRSIG no longer matches the received data. This demonstrates that resolver-side validation alone does not protect an ordinary stub.

```
[sanjanaboyina@Sanjanas-MacBook-Air phase6 % docker compose exec -T client delv -a /etc/bind/root.key @172.16.2.10 example.com
 ;; validating example.com/A: no valid signature found
 ;; RRSIG failed to verify resolving 'example.com/A/IN': 172.16.2.10#53
 ;; resolution failed: RRSIG failed to verify
 sanjanaboyina@Sanjanas-MacBook-Air phase6 %
```

Figure 27: DNSSEC validation failure for the forged last-mile response.

7 Phase 7: Encrypting the Last Mile with DNS-over-TLS

7.1 Objective

Phase 7 protects the client–resolver connection using DNS-over-TLS (DoT). The client connects to:

`resolver.cs6008.local:853`

The resolver certificate is signed by a private CA and contains the required DNS SAN `resolver.cs6008.local`. The client trusts this CA and does not use an insecure certificate bypass.

7.2 Deployment and Verification

Run from the Phase 7 directory:

```
cd dns_security/phase7
./scripts/sign-zones.sh
./scripts/create-pki.sh
docker compose build
docker compose up -d
./routers/configure-all-routes.sh
./scripts/install-ca.sh
./scripts/query-dot.sh
```

result:

- TLS handshake succeeds.
- The certificate validates for `resolver.cs6008.local`.
- DNS returns `172.16.1.20`.
- The DNSSEC authenticated-data flag is preserved.
- A packet capture shows TCP port 853 instead of cleartext DNS.

```
[sanjanaboyina@Sanjanas-MacBook-Air phase7 % ./routers/configure-all-routes.sh
 Static routes configured.
[sanjanaboyina@Sanjanas-MacBook-Air phase7 % ./scripts/install-ca.sh
 Updating certificates in /etc/ssl/certs...
 rehash: warning: skipping ca-certificates.crt,it does not contain exactly one certificate or CRL
 1 added, 0 removed; done.
 Running hooks in /etc/ca-certificates/update.d...
 done.
 CS6008 CA installed in the client trust store.
[sanjanaboyina@Sanjanas-MacBook-Air phase7 % ./scripts/query-dot.sh
 ;; TLS session (TLS1.3)-(ECDHE-SECP256R1)-(RSA-PSS-RSAE-SHA256)-(AES-256-GCM)
 ;; ->>HEADER<<- opcode: QUERY; status: NOERROR; id: 8232
 ;; Flags: qr rd ra ad; QUERY: 1; ANSWER: 1; AUTHORITY: 0; ADDITIONAL: 1

 ;; EDNS PSEUDOSECTION:
 ;; Version: 0; flags: ; UDP size: 1232 B; ext-rcode: NOERROR

 ;; QUESTION SECTION:
 ;; example.com.                IN      A

 ;; ANSWER SECTION:
 example.com.           86400   IN      A       172.16.1.20

 ;; Received 56 B
 ;; Time 2026-10-22 22:08:26 UTC
 ;; From 172.16.2.11@853(TLS) in 123.3 ms
 sanjanaboyina@Sanjanas-MacBook-Air phase7 %
```

Figure 28: Successful DNS-over-TLS resolution with certificate validation.

7.3 Attempt 1: Altering the Answer in Transit

The Phase 6 attacker operates on UDP/53 and cannot modify the encrypted DoT payload.

```
./scripts/setup-diversion.sh
./scripts/run-interceptor-forge.sh

# Strict client
./scripts/query-dot.sh

# Fallback-permitting client
./scripts/query-fallback.sh
```

result: both clients use DoT successfully. The forged address `172.16.1.30` is not accepted, and the fallback client does not use cleartext because DoT is still available.

```
sanjanaboyina@Sanjanas-MacBook-Air phase7 % ./scripts/setup-diversion.sh

Diversion route installed:
172.16.1.10 via 172.16.1.100 dev eth0 src 172.16.1.2 uid 0
    cache
[sanjanaboyina@Sanjanas-MacBook-Air phase7 % ./scripts/run-interceptor-forge.sh
Stopping any running interceptor...
Starting interceptor in FORGING mode (FORGE=1)...
Interface: eth0
Attacker MAC: 4a:08:ea:d0:ef:2f
Client MAC: fa:26:a8:b8:f6:f7
Mode: FORGING ENABLED
Listening for DNS responses destined for 172.16.1.10...
[sanjanaboyina@Sanjanas-MacBook-Air phase7 % ./scripts/query-dot.sh
;; TLS session (TLS1.3)-(ECDHE-SECP256R1)-(RSA-PSS-RSAE-SHA256)-(AES-256-GCM)
;; ->>HEADER<<- opcode: QUERY; status: NOERROR; id: 34233
;; Flags: qr rd ra ad; QUERY: 1; ANSWER: 1; AUTHORITY: 0; ADDITIONAL: 1

;; EDNS PSEUDOSECTION:
;; Version: 0; flags: ; UDP size: 1232 B; ext-rcode: NOERROR

;; QUESTION SECTION:
;; example.com.                 IN      A

;; ANSWER SECTION:
example.com.            86371   IN      A       172.16.1.20

;; Received 56 B
;; Time 2026-10-22 23:34:44 UTC
;; From 172.16.2.11@853(TLS) in 5.1 ms
[sanjanaboyina@Sanjanas-MacBook-Air phase7 % ./scripts/query-fallback.sh
Trying strict DoT first...
;; TLS session (TLS1.3)-(ECDHE-SECP256R1)-(RSA-PSS-RSAE-SHA256)-(AES-256-GCM)
;; ->>HEADER<<- opcode: QUERY; status: NOERROR; id: 7401
;; Flags: qr rd ra ad; QUERY: 1; ANSWER: 1; AUTHORITY: 0; ADDITIONAL: 1

;; EDNS PSEUDOSECTION:
;; Version: 0; flags: ; UDP size: 1232 B; ext-rcode: NOERROR

;; QUESTION SECTION:
;; example.com.                 IN      A

;; ANSWER SECTION:
example.com.            86365   IN      A       172.16.1.20

;; Received 56 B
;; Time 2026-10-22 23:34:50 UTC
;; From 172.16.2.11@853(TLS) in 4.2 ms
DoT succeeded; no cleartext fallback was needed.
sanjanaboyina@Sanjanas-MacBook-Air phase7 %
```

Figure 29: Answer-alteration attempt against both clients; no forged DoT answer is accepted.

7.4 Attempt 2: Rogue Resolver Certificate

The attacker replaces the valid certificate with an untrusted certificate using the correct hostname SAN.

```
./scripts/test-rogue-certificate.sh
```

result:

- The strict client rejects the certificate.
- The fallback-permitting client fails DoT and then uses cleartext.
- If the Phase 6 forger is still active, the fallback query may return `172.16.1.30`.

```
[sanjanaboyina@Sanjanas-MacBook-Air phase7 % ./scripts/test-rogue-certificate.sh
 [+] restart 0/1
  ⠿ Container dot-proxy Restarting                                    0.1s
Testing an untrusted certificate with the correct hostname SAN...
;; WARNING: TLS, handshake failed (Error in the certificate.)
;; ERROR: failed to query server resolver.cs6008.local@853(TCP)

What's next:
    Debug this Compose error with Gordon → docker ai "help me fix this compose error"
PASS: strict DoT validation rejected the rogue certificate.
Testing the fallback-permitting client with the same rogue certificate...
Trying strict DoT first...
;; WARNING: TLS, handshake failed (Error in the certificate.)
;; ERROR: failed to query server resolver.cs6008.local@853(TCP)

What's next:
    Debug this Compose error with Gordon → docker ai "help me fix this compose error"
DoT failed; falling back to cleartext DNS.

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> @172.16.2.10 example.com A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 41457
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: ac8e3db5ca19ac21010000006ada8ed2524cfd0115c82ec2 (good)
;; QUESTION SECTION:
;example.com.                   IN      A

;; ANSWER SECTION:
example.com.            86357   IN      A       172.16.1.30

;; Query time: 2 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Thu Oct 22 22:31:46 UTC 2026
;; MSG SIZE  rcvd: 84

PASS: fallback client used cleartext only after DoT certificate validation failed.
sanjanaboyina@Sanjanas-MacBook-Air phase7 %
```

Figure 30: Strict DoT validation rejecting the rogue certificate.

7.5 Attempt 3: Blocking the Encrypted Transport

```
./scripts/block-dot.sh

# Strict client: fails closed
./scripts/query-dot.sh

# Fallback-permitting client: uses cleartext after DoT fails
./scripts/query-fallback.sh

./scripts/unblock-dot.sh
```

result:

- The strict client receives no DNS answer.
- The fallback client resolves using cleartext DNS.
- Cleartext fallback loses DoT confidentiality and authentication.

```
[sanjanaboyina@Sanjanas-MacBook-Air phase7 % ./scripts/block-dot.sh
 TCP/853 blocked on edge-router.
[sanjanaboyina@Sanjanas-MacBook-Air phase7 % ./scripts/query-dot.sh
 ;; WARNING: connection timeout for 172.16.2.11@853(TLS)
 ;; ERROR: failed to query server resolver.cs6008.local@853(TCP)

 What's next:
     Debug this Compose error with Gordon → docker ai "help me fix this compose error"
 sanjanaboyina@Sanjanas-MacBook-Air phase7 %
```

Figure 31: Strict DoT client failing closed when TCP port 853 is blocked.

```
[sanjanaboyina@Sanjanas-MacBook-Air phase7 % ./scripts/query-fallback.sh
Trying strict DoT first...
;; WARNING: connection timeout for 172.16.2.11@853(TLS)
;; ERROR: failed to query server resolver.cs6008.local@853(TCP)

What's next:
    Debug this Compose error with Gordon → docker ai "help me fix this compose error"
DoT failed; falling back to cleartext DNS.

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> @172.16.2.10 example.com A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 10583
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 56edf71b7c874581010000006ada8fcf3f6ee0edf9c8d47b (good)
;; QUESTION SECTION:
;example.com.                   IN      A

;; ANSWER SECTION:
example.com.            86104   IN      A       172.16.1.30

;; Query time: 2 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Thu Oct 22 22:35:59 UTC 2026
;; MSG SIZE  rcvd: 84

sanjanaboyina@Sanjanas-MacBook-Air phase7 %
```

Figure 32: Fallback-permitting client resolving over cleartext after DoT fails.

7.6 Results

Attempt	Strict client	Fallback client
Alter answer	Forged answer rejected.	DoT succeeds; fallback is not triggered.
Rogue certificate	Certificate validation fails.	Falls back to cleartext.
Block TCP/853	Fails closed.	Falls back to cleartext.

7.7 DoT Compared with DoH

DoT was selected because it is simple to deploy and uses standard TLS on TCP port 853. DoH uses HTTPS on port 443 and can blend with ordinary web traffic, but requires an HTTP-compatible resolver endpoint and additional configuration. DoT protects DNS content on the client–resolver link but does not hide destination IP addresses, traffic timing, or the resolver's visibility into queries. It also does not protect the HTTP connection after DNS resolution.

7.8 Cleanup

```
./scripts/unblock-dot.sh
./scripts/stop-interceptor.sh
./scripts/teardown-diversion.sh
docker compose down
```